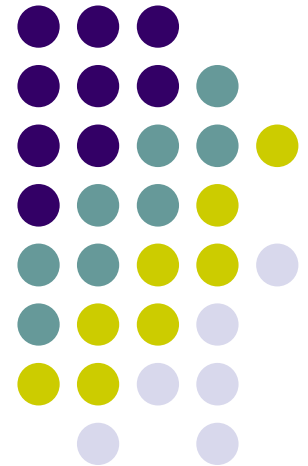
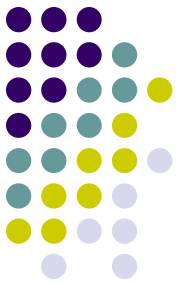


Flutter – Architecture of Flutter Application

Chapter 03



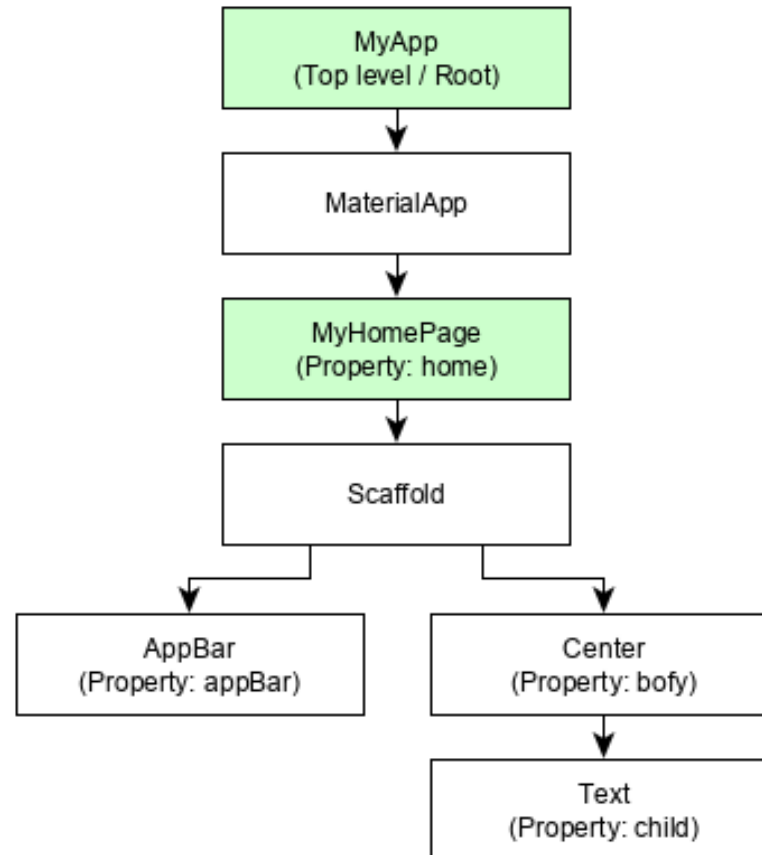
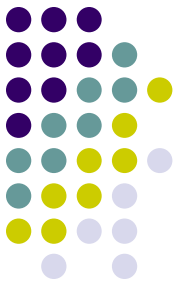


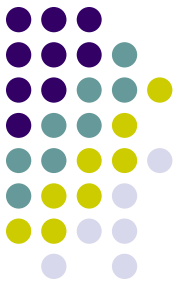
Widgets

The core concept of the Flutter framework is **In Flutter, Everything is a widget**. Widgets are basically user interface components used to create the user interface of the application.

- In *Flutter*, the application is itself a widget. The application is the top-level widget and its UI is built using one or more children (widgets), which again build using its children widgets. This **composability** feature helps us to create a user interface of any complexity.
- For example, the widget hierarchy of the hello world application (created in previous chapter) is as specified in the following diagram:

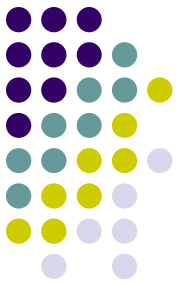
Widgets





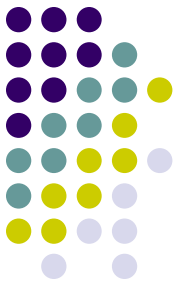
Widgets

- Here the following points are worth notable:
- **MyApp** is the user created widget and it is build using the Flutter native widget, **MaterialApp**.
- **MaterialApp** has a *home* property to specify the user interface of the home page, which is again a user created widget, *MyHomePage*.
- **MyHomePage** is build using another flutter native widget, *Scaffold*.
- **Scaffold** has two properties – *body* and *appBar*.
- **body** is used to specify its main user interface and *appBar* is used to specify its header user interface.
- **Header UI** is build using flutter native widget, *AppBar* and *Body UI* is build using *Center* widget.
- The **Center** widget has a property, *Child*, which refers the actual content and it is build using *Text* widget.



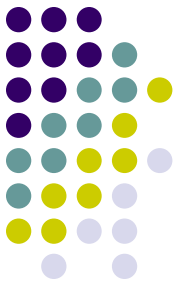
Gestures

- Flutter widgets support interaction through a special widget, GestureDetector.
- GestureDetector is an invisible widget having the ability to capture user interactions such as tapping, dragging, etc., of its child widget.
- Many native widgets of Flutter support interaction through the use of GestureDetector. We can also incorporate interactive feature into the existing widget by composing it with the GestureDetector widget.
- We will learn the gestures separately in the upcoming chapters.



Concept of State

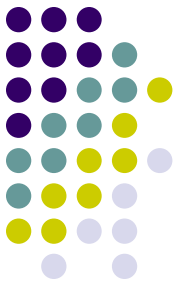
- Flutter widgets support State maintenance by providing a special widget, StatefulWidget.
- Widget needs to be derived from StatefulWidget widget to support state maintenance and all other widget should be derived from StatelessWidget.
- Flutter widgets are reactive in native. This is similar to reactjs and StatefulWidget will be auto re- rendered whenever its internal state is changed.
- The re-rendering is optimized by finding the difference between old and new widget UI and rendering only the necessary changes.



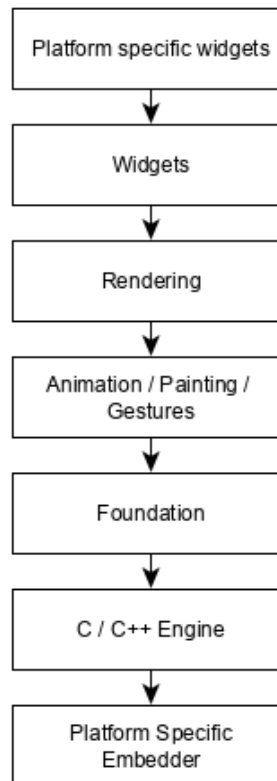
Layers

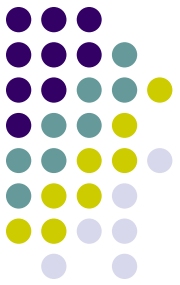
- The most important concept of Flutter framework is that the framework is grouped into multiple category in terms of complexity and clearly arranged in layers of decreasing complexity. A layer is build using its immediate next level layer. The top most layer is widget specific to *Android* and *iOS*. The next layer has all flutter native widgets. The next layer is *Rendering* layer, which is low level renderer component and renders everything in the flutter app. Layers goes down to core platform specific code.

Layers



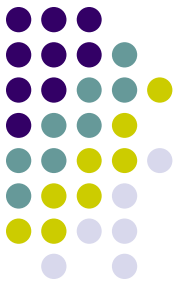
- The general overview of a layer in Flutter is specified in the below diagram:





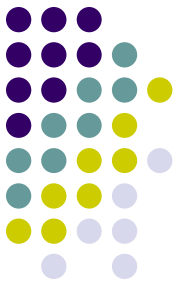
Layers

- The following points summarize the architecture of Flutter:
- In Flutter, everything is a widget and a complex widget is composed of already existing widgets.
- Interactive features can be incorporated whenever necessary using *GestureDetector* widget.
- The state of a widget can be maintained whenever necessary using *StatefulWidget* widget.
- Flutter offers layered design so that any layer can be programmed depending on the complexity of the task.



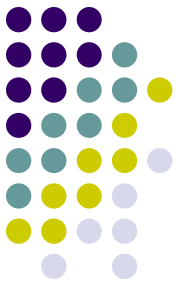
Introduction to Widgets

- As we learned in the earlier chapter, widgets are everything in Flutter framework. We have already learned how to create new widgets in previous chapters.
- In this chapter, let us understand the actual concept behind creating the widgets and the different type of widgets available in *Flutter* framework.
- Let us check the *Hello World* application's *MyHomePage* widget. The code for this purpose is as given below:



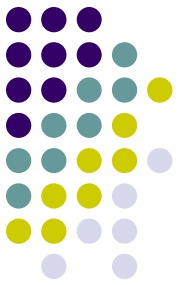
Introduction to Widgets

```
class MyHomePage extends StatelessWidget
{
  MyHomePage(
    {
      Key key,
      this.title}
    ) : super(key: key); final String title; @override Widget
  build(BuildContext context) { return Scaffold( appBar:
  AppBar( title: Text(this.title), ), body: Center( child:
  Text( 'Hello World',
  )),
  );
}
```



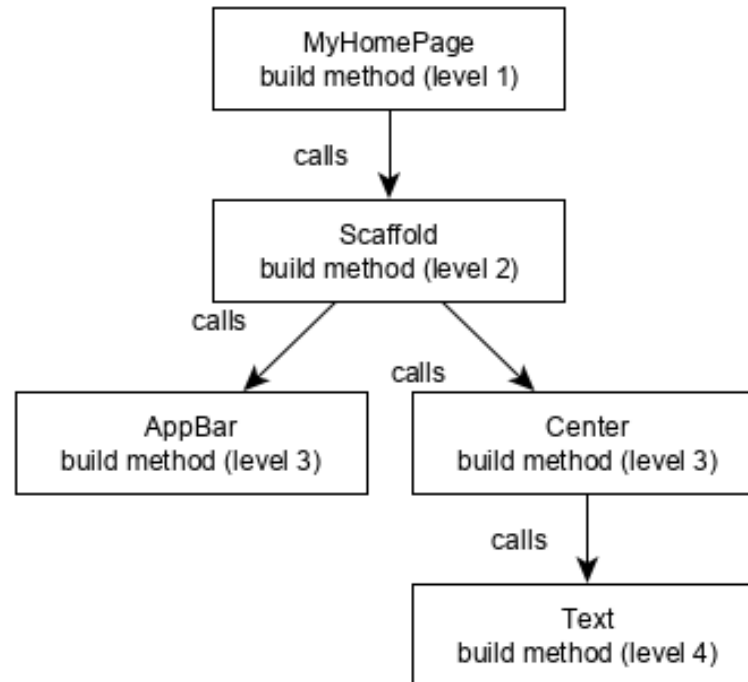
Introduction to Widgets

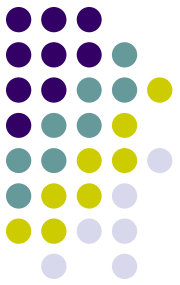
- Here, we have created a new widget by extending *StatelessWidget*.
- Note that the *StatelessWidget* only requires a single method *build* to be implemented in its derived class. The *build* method gets the context environment necessary to build the widgets through *BuildContext* parameter and returns the widget it builds.
- In the code, we have used *title* as one of the constructor argument and also used *Key* as another argument. The *title* is used to display the title and *Key* is used to identify the widget in the build environment.
- Here, the *build* method calls the *build* method of *Scaffold*, which in turn calls the *build* method of *AppBar* and *Center* to build its user interface.
- Finally, *Center* build method calls *Text* build method.



Introduction to Widgets

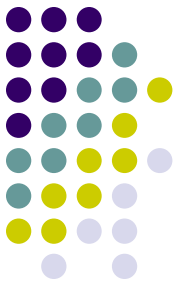
- For a better understanding, the visual representation of the same is given below:





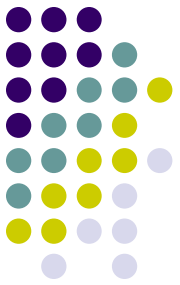
Widget Build Visualization

- In *Flutter*, widgets can be grouped into multiple categories based on their features, as listed below:
- Platform specific widgets
- Layout widgets
- State maintenance widgets
- Platform independent / basic widgets



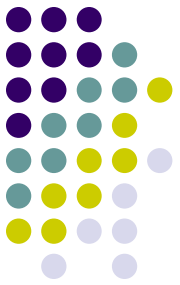
Platform specific widgets

- Flutter has widgets specific to a particular platform - Android or iOS.
- Android specific widgets are designed in accordance with *Material design guideline* by Android OS. Android specific widgets are called as *Material widgets*.
- iOS specific widgets are designed in accordance with *Human Interface Guidelines* by Apple and they are called as *Cupertino widgets*.



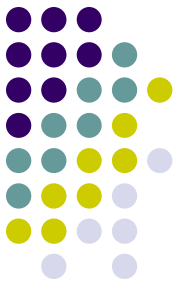
Platform specific widgets

- Some of the most used material widgets are as follows:
- Scaffold
- AppBar
- BottomNavigationBar
- TabBar
- TabBarView
- ListTile
- RaisedButton
- FloatingActionButton
- FlatButton
- IconButton
- DropdownButton
- PopupMenuButton
- ButtonBar
- TextField
- Checkbox
- Radio
- Switch
- Slider
- Date & Time Pickers
- SimpleDialog
- AlertDialog



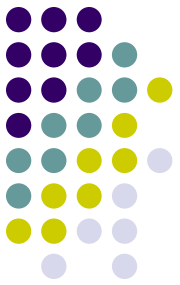
Platform specific widgets

- Some of the most used *Cupertino* widgets are as follows:
- CupertinoButton
- CupertinoPicker
- CupertinoDatePicker
- CupertinoTimerPicker
- CupertinoNavigationBar
- CupertinoTabBar
- CupertinoTabScaffold
- CupertinoTabView
- CupertinoTextField
- CupertinoDialog
- CupertinoDialogAction
- CupertinoFullscreenDialogTransition
- CupertinoPageScaffold
- CupertinoPageTransition
- CupertinoActionSheet
- CupertinoActivityIndicator
- CupertinoAlertDialog
- CupertinoPopupSurface



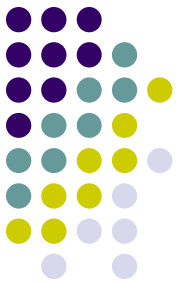
Layout widgets

- In Flutter, a widget can be created by composing one or more widgets. To compose multiple widgets into a single widget, *Flutter* provides large number of widgets with layout feature. For example, the child widget can be centered using *Center* widget.
- Some of the popular layout widgets are as follows:
- Container: A rectangular box decorated using *BoxDecoration* widgets with background, border and shadow.
- Center: Center its child widget
- Row: Arrange its children in the horizontal direction.
- Column: Arrange its children in the vertical direction.
- Stack: Arrange one above the another.
- We will check the layout widgets in detail in the upcoming *Introduction to layout widgets* chapter.



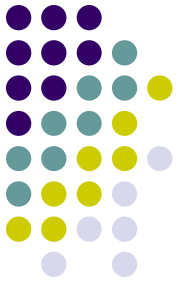
State maintenance widgets

- In Flutter, all widgets are either derived from *StatelessWidget* or *StatefulWidget*.
- Widget derived from *StatelessWidget* does not have any state information but it may contain widget derived from *StatefulWidget*. The dynamic nature of the application is through interactive behavior of the widgets and the state changes during interaction. For example, tapping a counter button will increase / decrease the internal state of the counter by one and reactive nature of the *Flutter* widget will auto re-render the widget using new state information.
- We will learn the concept of *StatefulWidget* widgets in detail in the upcoming *State management* chapter.



Platform independent / basic widgets

- *Flutter* provides large number of basic widgets to create simple as well as complex user interface in a platform independent manner. Let us see some of the basic widgets in this chapter.



Thank You